

FPGA Implementation of Spiking Neural Networks for Efficient Digit Recognition

A Hardware-Software Co-Design Approach

Soham Vaishnav, Abhinav S, Rupak Antani

International Institute of Information Technology, Hyderabad

December 4, 2025

Outline

- 1 Introduction & Motivation
- 2 Theoretical Background
- 3 Software Training & Architecture
- 4 Hardware Implementation (RTL)
- 5 Results & Analysis
- 6 Conclusion

The Energy Crisis in AI

The Problem:

- Traditional Deep Neural Networks (DNNs/CNNs) are computationally expensive.
- Von Neumann bottleneck: High energy cost due to data movement.
- Training consumes 100–300W on GPUs.
- Unsuitable for battery-powered edge devices.

The Solution: Spiking Neural Networks (SNNs)

- **Bio-inspired:** Mimics the brain's energy efficiency.
- **Event-driven:** Computation occurs *only* when spikes occur.
- **Sparsity:** Silent neurons consume negligible power.
- **Goal:** Implement an end-to-end SNN for digit classification on a low-power FPGA.

Comparison: CNN vs. SNN

Table: Architectural Differences

Feature	CNN (Traditional)	SNN (Our Work)
Data Type	Static Frames	Temporal Spikes
Information	Real-valued	Binary Events (0/1)
Computation	Dense MACs	Event-driven Accumulation
Training	Backpropagation	Surrogate Gradient
Hardware	GPU/TPU	Neuromorphic/FPGA
Energy	High (Continuous)	Low (Sparse)

Leaky Integrate-and-Fire (LIF) Model

We utilize the LIF model for its balance of biological plausibility and hardware efficiency.

Mathematical Model

$$U_i[t] = \beta U_i[t - 1] + \sum_j w_{ij} s_j[t] - U_{th,i} \cdot \text{spike}_i[t - 1] \quad (1)$$

- β : Decay factor (leaks charge over time).
- w_{ij} : Synaptic weight.
- $s_j[t]$: Input binary spike.
- **Hardware Advantage:** No complex multiplications required during inference; operations are reduced to additions and bit-shifts.

Rate Encoding Strategy

Converting Static Images to Spikes:

- MNIST images are static (28×28 pixels).
- **Rate Coding:** Pixel intensity maps to spike probability.
- Higher intensity = Higher firing frequency.
- Inference Window: 25 timesteps.

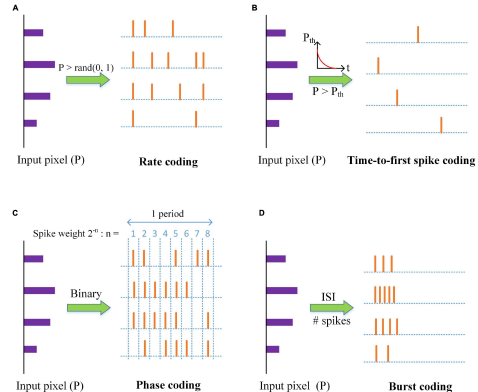


Figure: Visualization of pixel intensity to spike train conversion.

SNN Architecture

Software Model (PyTorch):

- Input: 784 neurons.
- Hidden 1: 1024 neurons.
- Hidden 2: 512 neurons.
- Output: 10 neurons.

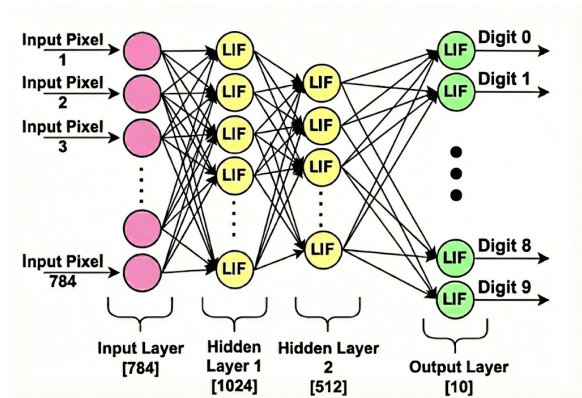
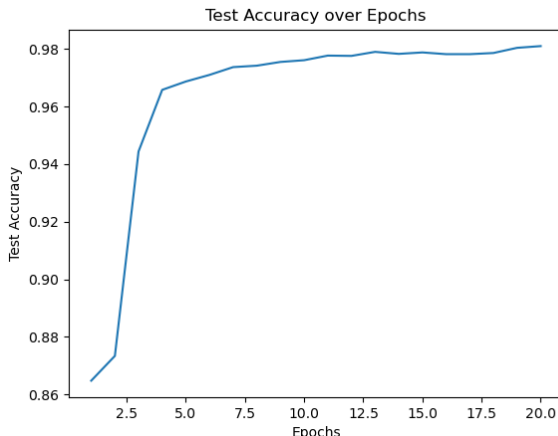


Figure: 3-Layer Fully Connected Architecture.

SNN Architecture

- **Accuracy:** 98.1% (Software).
- **Training Method:** Surrogate Gradient Descent (Fast Sigmoid) to overcome non-differentiable spike generation.



Challenge: The full software model is too large for the target Intel Cyclone V FPGA.

Model Downsizing

We optimized the model specifically for hardware deployment:

- Input downsampled to 14×14 (196 neurons).
- Hidden layers reduced to 256 and 128 neurons.
- **Quantization:** 32-bit Float $\rightarrow$ 16-bit Fixed Point.
- **Result:** Model fits in 331KB memory with only **1.14%** accuracy loss.

Design Style: Modular Verilog HDL controlled by a central FSM.

- ① **Rate Encoder:** Converts pixel data to spikes on-the-fly.
- ② **Time-Multiplexed Neurons:**
 - Instead of instantiating physical logic for every neuron, we use a **single pipeline** per layer.
 - Neurons are processed sequentially, saving massive area.
- ③ **Spike-Count Classifier:** Output determined by the neuron with maximum spike count over the time window.

Synthesis Results (Intel Cyclone V)

Performance Metrics:

- **Max Clock Frequency** = 72.98MHz
- **Throughput:** ≈ 50 images/sec
- **Setup Slack** = 12.88 ns
- **Hold Slack** = 0.125 ns

Table: FPGA Resource Utilization (Cyclone V)

Resource	Used	Utilization (%)
Logic Elements (ALMs)	8,152	7%
Registers	14,526	–
Memory Bits	1,347,584	11%
DSP Blocks	2	< 1%

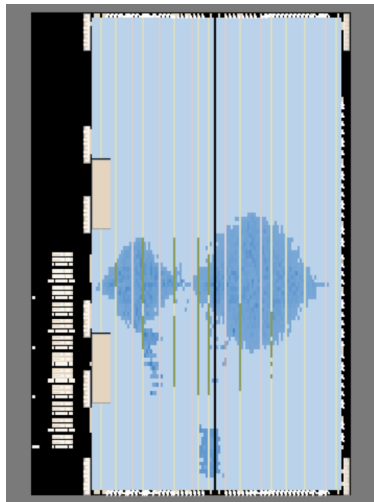


Figure: FPGA Chip Plan.

Comparison

Work	Platform	Neurons	Resource Util	Frequency
Diehl et al. [1]	Software	784-1200-1200-10	N/A	N/A
Farsa et al. [2]	Virtex-6	25-5-1	N/A	189 MHz
Ali et al. [3]	Artix-7	4096-512-2	31.4 % LUT + 4.3 % FF	67 MHz
Han et al. [4]	FPGA	784-1024-1024-10	2.46 % LUT + 1.67 % FF	67 MHz
Kuang et al. [5]	Stratix III	N/A	23 % LUT + 4 % FF	N/A
This Work	Cyclone V	196-256-128-10	7 % LUT + 2 % DSP	72.98 MHz

Key Takeaway: We achieve software-comparable accuracy with minimal hardware resources (<10% utilization).

Conclusion & Future Directions

Summary:

- Successfully implemented end-to-end SNN accelerator on FPGA Simulator.
- Demonstrated **computational and memory efficiency** as compared to conventional CNNs in basic image classification tasks.
- Bridged the gap between PyTorch training and Verilog deployment.

Future Work:

- SNNs for temporal and sparse processing – ECG, etc. due to their inherent time-dependent processing
- **On-Chip Learning:** Implementing STDP for real-time adaptability.
- **Convolutional SNNs:** To scale to complex datasets (CIFAR-10).

Thank You

Questions?

soham.vaishnav@research.iiit.ac.in
abhinav.s@students.iiit.ac.in
rupak.antani@students.iiit.ac.in

References I



P. U. Diehl, D. Neil, J. Binas, M. Cook, S.-C. Liu, and M. Pfeiffer, "Fast-classifying, high-accuracy spiking deep networks through weight and threshold balancing," in *Proc. International Joint Conference on Neural Networks (IJCNN)*, 2015, pp. 1–8.



E. Z. Farsa, A. Ahmadi, M. A. Maleki, M. Gholami, and H. N. Rad, "A low-cost high-speed neuromorphic hardware based on spiking neural network," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 66, no. 9, pp. 1582–1586, 2019.



A. H. Ali, M. Navardi, and T. Mohsenin, "Energy-aware FPGA implementation of spiking neural network with LIF neurons," *arXiv preprint arXiv:2411.01628*, 2024.



J. Han, Z. Li, W. Zheng, and Y. Zhang, "Hardware implementation of spiking neural networks on FPGA," *Tsinghua Science and Technology*, vol. 25, no. 4, pp. 479–486, 2020.



Z. Kuang, J. Wang, S. Yang, G. Yi, B. Deng, and X. Wei, "Digital implementation of the spiking neural network and its digit recognition," in *Proc. 2019 Chinese Control and Decision Conference (CCDC)*, 2019, pp. 3621–3625.